With the *-c* operation, *losetup* will detect the increase in the size but you still need to make the change in the file system for the loop device.

```
$resize2fs /dev/loop0
```

With this, the new resized storage is ready to be used.

Decreasing the storage: To decrease the partition size, it is not required to decrease the size of the file with the 'truncate' utility, since it may leave some half data chunks or bad blocks, and due to these bad blocks you will not be able to use it until you format it again. So, the best solution can be to just decrease the file system layer on the file using the *resize2fs* command:

```
$resize2fs test.img 1G
```

…where 1G is the new decreased size. You do not need to worry about the size shown in metadata since the user will only be able to use the formatted space. Since the file is a sparse file, the remaining space will not take any space in your system. In the looped device, we just need to detect the changed size, as follows:

```
$losetup -c /dev/loop0
```

Now, your storage of decreased size is ready.

Tip: To detach the storage from the loop device use the following command:

```
$ losetup -d /dev/loop0
```

Backing up your data: Features that every storage service provider must have are backup, snapshot and clustering. Data snapshots seem to be the less sensible option, so let's not concentrate on them and discuss the topic at a later stage. But talking of backing up data, this feature creates a backup file which saves all the useful data for future use.

So let's take a backup, which can be successfully done by using the *rsync* utility:

```
$rsync -avz test.img test_backup.img
```

This will create a *test_backup.img* file with the same data blocks. So every time you run this command, new changes made since the last backup will get saved in *test_backup.img*.

Warning:

If you create the backup image of your file system through *rsync*, the backup file will not be a sparse file; so it will allocate all of the space at once.

Although this looks simple, it is not a very efficient solution since once data starts overwriting in some blocks in the original storage image, it will get copied exactly in the backup image.

So a better solution can be to *rsync* the mounted path:

```
$rsync -avz /mnt/ /mnt_backup
```

To make the backup process automatic, you can use *lsyncd* for live synchronisation. Install *lsyncd* in an RHEL system, as follows:

```
$yum install -y lsyncd
```

Edit the configuration file *lsyncd.conf*, as follows:

```
$cat /etc/lsyncd.conf
----
-- User configuration file for lsyncd.
--
-- Simple example for default rsync.
--
settings = {
 logfile = "/var/log/lsyncd.log",
 statusFile = "/var/log/lsyncd.stat",
 statusInterval = 2,
}
sync{
 default.rsync,
 source="/mnt/",
 target="192.168.1.15:/backup/",
 rsync={rsh ="/usr/bin/ssh -l root -i /root/.ssh/id_rsa",}
}
```

Note: You need to first connect to the backup machine by ssh, using *ssh-keygen*.

If you want to automate the backup process, you can use *fsmonitor npm*, as follows:

```
$fsmonitor rsync -azP /mnt/ /mnt_backup
```

You can even use *rsnapshot* for incremental backup.

Snapshots of the storage data: Snapshot is a facility available in different Linux distros to save your storage at particular stages. It can be a useful feature for STaaS providers, because it will help your storage to revert back to any previous state. Snapshot is different from backup because it doesn't take up your storage space until changes are made in the storage. To save space, it just copies the files that have been deleted. For our file storage, we will be using the *qemu-img* utility for a snapshot. So first create a new snapshot of your storage, as shown below:

```
$qemu-img snapshot -c backup_snapshot test.img
```